

GY-86

1. I2C

软件实现配置流程

1. 随便选两个引脚，一个为SCL，一个为SDA，最好define一下SDA，SCL，引脚一定要初始化为开漏输出模式

为什么要使用开漏输出？

- 如果 GPIO 用推挽输出，一个设备输出高，另一个设备输出低，就会形成 VCC 到 GND 的低阻通路，大电流，可能烧引脚。
- 满足I2C这样的条件：任何一个设备拉低，总线就是低电平，所有设备都释放，总线才被上拉拉高。
- 开漏输出下，SDA就是双向的，“输出其实也可以当输入用”：开漏输出写 1 时，引脚相当于“释放”了，外部从机可以把它拉低。此时输入缓冲器仍然可以读引脚电平，于是就能“输出模式下读输入”。

2. 写读SDA，SCL的代码，
很轻松，就是将GPIO的电平置零，置一，不多赘述，要理解的是后面的时序基本单元，很重要
3. 完成I2C六个时序基本单元：启动，停止，数据发送，数据接收，接收ack，发送ack

六个时序

默认来讲，SCL和SDA两条线均由主机操控，只有主机释放时，从机才有可能拉低SDA或SCL

- 起始条件：SCL高电平期间，SDA从高电平切换 到低电平

```
void MyI2C_Start()
{
    MyI2C_W_SDA(1);
    MyI2C_W_SCL(1);
    MyI2C_W_SDA(0);
    MyI2C_W_SCL(0); //注意此时要将SCL再降下来，因为接下来要开始传数据了（改变SDA时SCL一定为低电平）
}
```

不妨思考一下这里为什么要**先确保SDA为1，再去升高SCL**？

解释：若此时SDA为0，如果先升高SCL，接下来你又升高SDA为1，此时会被识别为终止信号，而先确保SDA为1就可以避免这种信号的错误识别。

- 终止条件：SCL高电平期间，SDA从低电平切换到高电平

```
void MyI2C_Stop()
{
    MyI2C_W_SDA(0);
    MyI2C_W_SCL(1);
    MyI2C_W_SDA(1);
}
```

那么结合我刚刚的问题再看一下终止条件的代码，为什么要**先确保SDA为1，再去升高SCL**？相信你已经很明白了

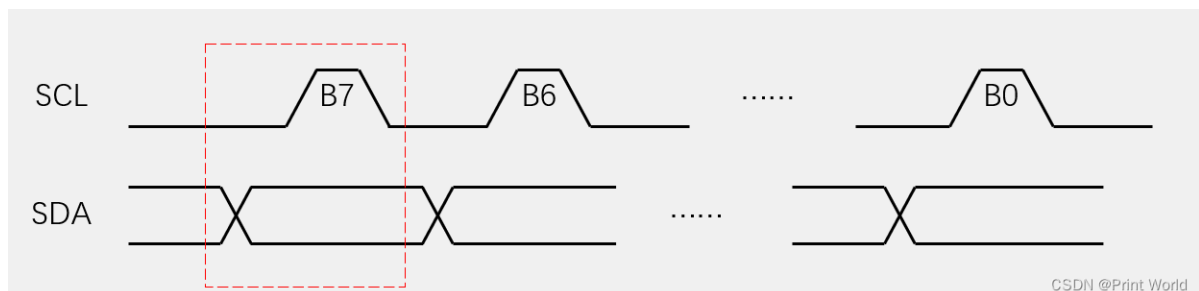


起始终止时序图

• 发送一字节数据：在SCL低电位期间，主机将数据放到SDA线上（高位先行），然后释放SCL，从机将在SCL高电平期间读取一字节数据

注：SCL高电平期间SDA不允许有数据变化！

```
void MyI2C_SendByte(uint8_t Byte)
{
    for(uint8_t i = 0; i < 8; i++)
    {
        MyI2C_W_SDA(Byte & (0x80 >> i)); //从高位到低位开始读取
        MyI2C_W_SCL(1); //这里置高电平，让从机在SDA读取数据
        MyI2C_W_SCL(0);
    }
}
```



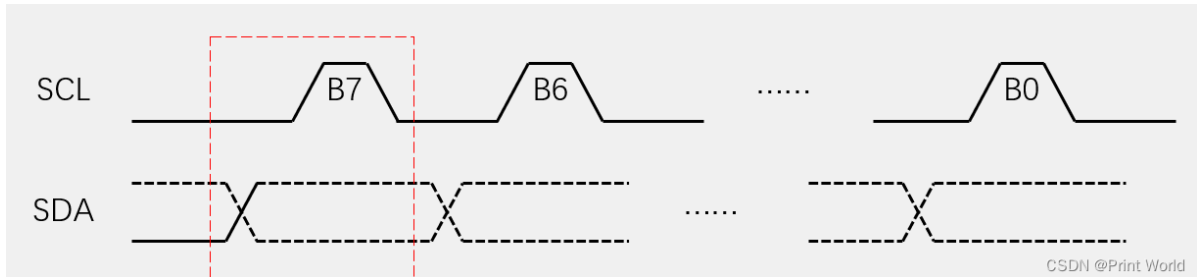
• 接受一位数据：在SCL低电位期间，从机将数据放到SDA线上（高位先行），然后释放SCL，主机将在SCL高电平期间读取一位数据

注：主机接收前需要先释放SDA

```

uint8_t MyI2C_ReceiveByte()
{
    uint8_t Byte = 0x00;
    MyI2C_W_SDA(1); //释放SDA, 让从机来控制SDA
    for(uint8_t i = 0; i < 8; i++)
    {
        MyI2C_W_SCL(1);
        if(MyI2C_R_SDA() == 1) Byte |= (0x80 >> i);
        MyI2C_W_SCL(0);
    }
    return Byte;
}

```



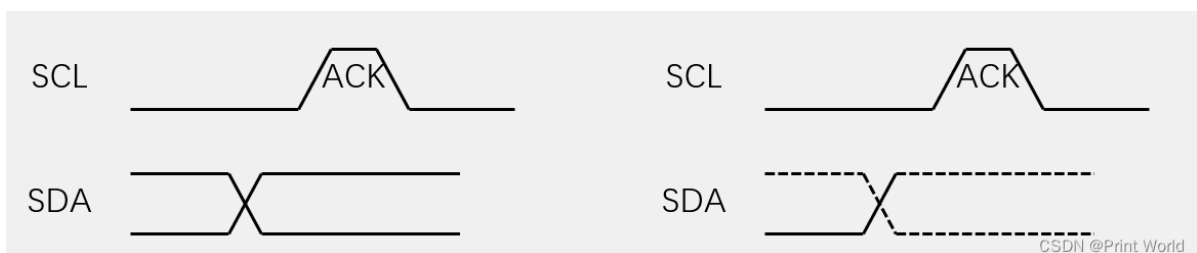
- 发送应答位：在数据接收完成后，在下一个时钟时发送一位数据，0表示应答，1表示未应答
- 接收应答位：在数据发送完成后，在下一个时钟时接收一位数据，判断从机是否应答，在接收前，主机需要释放SDA

```

void MyI2C_SendAck(uint8_t BitValue)
{
    MyI2C_W_SDA(BitValue);
    MyI2C_W_SCL(1);
    Delay_us(5);
    MyI2C_W_SCL(0);
}

uint8_t MyI2C_ReceiveAck()
{
    uint8_t Ack;
    MyI2C_W_SDA(1);
    MyI2C_W_SCL(1);
    Ack = MyI2C_R_SDA();
    MyI2C_W_SCL(0);
    return Ack;
}

```



指定地址读写

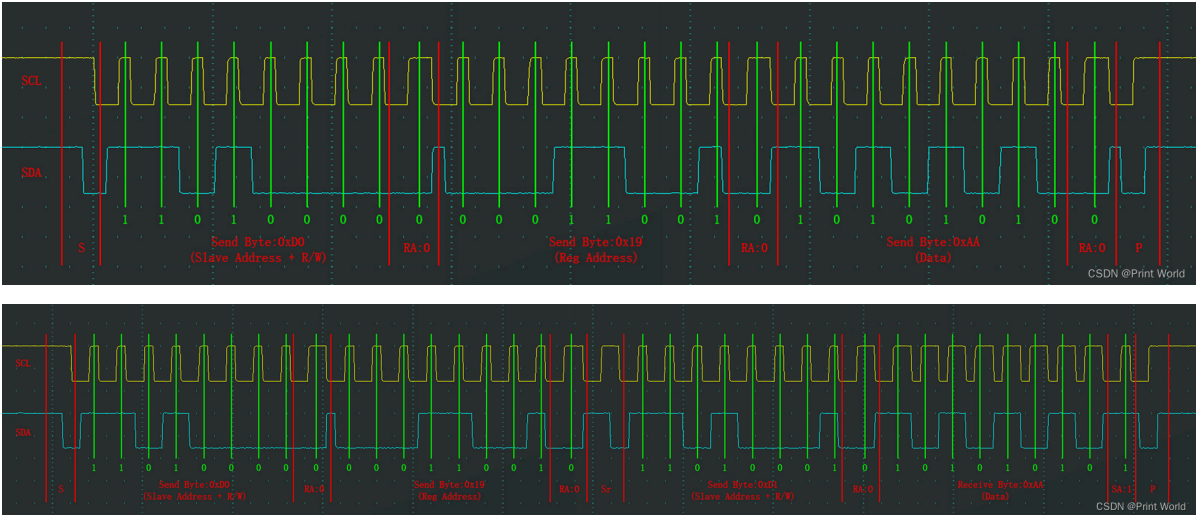
本质就是将上述六个时序按照合理顺序进行。

写：开始->设备地址（写）->接收应答->寄存器地址->接收应答->x字节数据->接收应答->终止

读：开始 -> 设备地址（写） -> 接收应答 -> 寄存器地址 -> 接收应答 -> 再开始 -> 设备地址（读） -> 接收应答 -> 接收x字节数据 -> 发送应答 -> 终止

为什么读操作要“先写”？

因为I2C总线没有独立的“寄存器地址线”。从机内部有一个“寄存器指针”。主机必须先通过一次“写操作”（发写地址+寄存器地址），把从机内部的指针拨到目标寄存器上，然后再通过“重复起始条件”切换成“读操作”（发读地址），从机才会把指针指向的数据吐出来。（相当于移动指针的必要行为）



你知道上面两个图分别是什么操作吗？

硬件实现配置流程（略）

2.串口通信

基础概念

- 波特率：串口通信速率。
- 起始位：标志一个数据帧开始，固定低电平。
- 数据位：有效载荷，低位先行。
- 校验位：用于数据验证，根据数据位计算。
- 停止位：数据帧间隔，固定高电平。

配置流程

1. 开启 GPIO 和 USART 时钟

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | RCC_APB2Periph_USART1, ENABLE);
```

2. 配置 GPIO

- TX: 复用推挽输出。就是将引脚的控制权从普通的GPIO控制器（GPIO_OCTL寄存器）移交给片上外设，这里就是USART
- RX: 浮空输入或上拉输入。

```
GPIO_InitTypeDef GPIO_InitStructure;

// TX PA9
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOA, &GPIO_InitStructure);

// RX PA10
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
GPIO_Init(GPIOA, &GPIO_InitStructure);
```

3. 配置 USART

```
typedef struct
{
    uint32_t USART_BaudRate;           // 波特率
    uint16_t USART_WordLength;         // 字长
    uint16_t USART_StopBits;           // 停止位
    uint16_t USART_Parity;              // 校验位
    uint16_t USART_Mode;                // 接收/发送
    uint16_t USART_HardwareFlowControl; // 硬件流控
} USART_InitTypeDef;
```

例:

```
USART_InitTypeDef USART_InitStructure;
USART_InitStructure.USART_BaudRate = 9600;
USART_InitStructure.USART_WordLength = USART_WordLength_8b;
USART_InitStructure.USART_StopBits = USART_StopBits_1;
USART_InitStructure.USART_Parity = USART_Parity_No;
USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;
USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
USART_Init(USART1, &USART_InitStructure);

while (1)
{
    USART_Cmd(USART1, ENABLE);
```

4. 发送数据

```
void Serial_Send_Byte(uint8_t byte)
{
    USART_SendData(USART1, byte);
    while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);
}
```

注意:

- TXE: 发送数据寄存器空，可以写下一个字节。

- **TC**: 发送完成，整个帧发送完毕。

5. (可选) 接收数据，配置中断

如果你说你连中断都不知道是什么，那我在这里解释一下

CPU在正常执行主程序（如 `while(1)` 里的代码）时，突然发生了一个紧急事件（如串口收到了数据、定时器时间到了），CPU**暂停**当前工作，**跳转**去执行处理这个事件的代码（中断服务函数），处理完后再**回到原来的地方**继续执行。

中断方式：主程序正常跑。串口收到数据了，硬件自动触发中断，CPU才过来把数据读走。读完之后继续干自己的事。这就是**异步**处理，效率极高。

中断的四大核心要素（以STM32为例）

1. 中断源：

谁触发了中断？可能是串口收到了数据、I2C完成了发送、定时器溢出了、按键按下了。

2. NVIC：

单片机的“**中断大管家**”。它负责**管理所有中断**的开启/关闭、优先级**，以及决定当前该让哪个中断先执行。

3. 中断向量表：

一张存储在Flash固定位置的表格。当某个中断触发时，CPU通过查表找到对应的**中断服务函数 (ISR)** 的入口地址。

4. **中断服务函数：

你自己写的处理代码。比如 `void USART1_IRQHandler(void)`。

回归正题：

使能USART1的接收数据寄存器非空中断（RXNE）：

```
USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
```

配置 NVIC：

```
NVIC_InitTypeDef NVIC_InitStructure;
NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init(&NVIC_InitStructure);
```

中断服务函数：

```

void USART1_IRQHandler(void)
{
    if (USART_GetITStatus(USART1, USART_IT_RXNE) != RESET)
    {
        uint8_t data = USART_ReceiveData(USART1);
        // 处理数据
        USART_ClearITPendingBit(USART1, USART_IT_RXNE);
    }
}

```

###

完成了上述的知识准备工作，我们就可以着手GY86的三个模块了。

3.MPU6050-惯性测量单元

首先是惯性测量单元，这玩意可以测量芯片的加速度和角速度，来翻翻数据手册,主要用到的就是我划线的部分

4	REGISTER DESCRIPTIONS	寄存器描述
4.1	REGISTERS 13 TO 16 - SELF TEST REGISTERS	自检寄存器
4.2	REGISTER 25 - SAMPLE RATE DIVIDER	采样频率分频器
4.3	REGISTER 26 - CONFIGURATION	配置
4.4	REGISTER 27 - GYROSCOPE CONFIGURATION	陀螺仪配置
4.5	REGISTER 28 - ACCELEROMETER CONFIGURATION	加速度计配置
4.6	REGISTER 31 - MOTION DETECTION THRESHOLD	运动检测阈值
4.7	REGISTER 35 - FIFO ENABLE	FIFO 使能
4.8	REGISTER 36 - I2C MASTER CONTROL	I2C 主机控制
4.9	REGISTERS 37 TO 39 - I2C SLAVE 0 CONTROL	I2C 从机 0 控制
4.10	REGISTERS 40 TO 42 - I2C SLAVE 1 CONTROL	I2C 从机 1 控制
4.11	REGISTERS 43 TO 45 - I2C SLAVE 2 CONTROL	I2C 从机 2 控制
4.12	REGISTERS 46 TO 48 - I2C SLAVE 3 CONTROL	I2C 从机 3 控制
4.13	REGISTERS 49 TO 53 - I2C SLAVE 4 CONTROL	I2C 从机 4 控制
4.14	REGISTER 54 - I2C MASTER STATUS	I2C 主机状态
4.15	REGISTER 55 - INT PIN / BYPASS ENABLE CONFIGURATION	INT 引脚/旁路有效 使能配置
4.16	REGISTER 56 - INTERRUPT ENABLE	中断使能
4.17	REGISTER 58 - INTERRUPT STATUS	中断状态
4.18	REGISTERS 59 TO 64 - ACCELEROMETER MEASUREMENTS	加速度计测量值
4.19	REGISTERS 65 AND 66 - TEMPERATURE MEASUREMENT	温度测量值
4.20	REGISTERS 67 TO 72 - GYROSCOPE MEASUREMENTS	陀螺仪测量值
4.21	REGISTERS 73 TO 96 - EXTERNAL SENSOR DATA	外部传感器数据
4.22	REGISTER 99 - I2C SLAVE 0 DATA OUT	I2C 从机 0 数据输出
4.23	REGISTER 100 - I2C SLAVE 1 DATA OUT	I2C 从机 1 数据输出
4.24	REGISTER 101 - I2C SLAVE 2 DATA OUT	I2C 从机 2 数据输出
4.25	REGISTER 102 - I2C SLAVE 3 DATA OUT	I2C 从机 3 数据输出
4.26	REGISTER 103 - I2C MASTER DELAY CONTROL	I2C 主机延时管理

4.29	REGISTER 106	-	MOTION DETECTION CONTROL	运动检测控制
4.29	REGISTER 106	-	USER CONTROL	用户配置
4.30	REGISTER 107	-	POWER MANAGEMENT 1	电源管理 1
4.31	REGISTER 108	-	POWER MANAGEMENT 2	电源管理 2
4.32	REGISTER 114 AND 115	-	FIFO COUNT REGISTERS	FIFO 计数寄存器
4.33	REGISTER 116	-	FIFO READ WRITE	FIFO 读写
4.34	REGISTER 117	-	WHO AM I	我是谁

读写代码

先来写读写MPU6050的代码，如果你理解了I2C那么就是手到擒来，不再说明了。后面几个芯片我也不写了。

```
uint8_t MPU6050_WriteReg(uint8_t regAddr, uint8_t data)
{
    uint8_t ack;
    MyI2C_Start();

    MyI2C_SendByte(MPU6050_ADDR);
    ack = MyI2C_ReceiveAck();
    if(ack == 1)
    {
        MyI2C_Stop();
        return 1;
    }

    MyI2C_SendByte(regAddr);
    ack = MyI2C_ReceiveAck();
    if(ack == 1)
    {
        MyI2C_Stop();
        return 1;
    }

    MyI2C_SendByte(data);
    ack = MyI2C_ReceiveAck();
    if(ack == 1)
    {
        MyI2C_Stop();
        return 1;
    }

    MyI2C_Stop();
    return 0;
}
//这里返回值不是data，注意了
uint8_t MPU6050_ReadReg(uint8_t regAddr, uint8_t *pData)
{
    uint8_t ack;
    MyI2C_Start();

    MyI2C_SendByte(MPU6050_ADDR);
    ack = MyI2C_ReceiveAck();
```

```

    if(ack == 1)
    {
        MyI2C_Stop();
        return 1;
    }

    MyI2C_SendByte(regAddr);
    ack = MyI2C_ReceiveAck();
    if(ack == 1)
    {
        MyI2C_Stop();
        return 1;
    }

    MyI2C_Start();

    MyI2C_SendByte(MPU6050_ADDR | 1);
    ack = MyI2C_ReceiveAck();
    if(ack == 1)
    {
        MyI2C_Stop();
        return 1;
    }

    *pData = MyI2C_ReceiveByte();
    MyI2C_SendAck(1);

    MyI2C_Stop();
    return 0;
}

```

初始化MPU6050

```

void MPU6050_Init()
{
    MyI2C_Init();
    MPU6050_WriteReg(MPU6050_PWR_MGMT_1, 0x00);
    MPU6050_WriteReg(MPU6050_PWR_MGMT_2, 0x00);
    MPU6050_WriteReg(MPU6050_SMPLRT_DIV, 0x09);
    MPU6050_WriteReg(MPU6050_CONFIG, 0x06);
    MPU6050_WriteReg(MPU6050_GYRO_CONFIG, 0x18);
    MPU6050_WriteReg(MPU6050_ACCEL_CONFIG, 0x18);
}

```

1. MPU6050_PWR_MGMT_1

电源管理寄存器 1，芯片开关

- **Bit 7 (DEVICE_RESET) = 0**：不复位芯片（我们不需要在这里软件复位，因为已经手动上电了）。
- **Bit 6 (SLEEP) = 0**：**解除睡眠**。这是让芯片开始工作的关键，之前的全 0 故障就是卡在这里。
- **Bit 5 (CYCLE) = 0**：关闭循环唤醒模式（正常连续工作）。

- Bit 4 (TEMP_DIS) = 0: 使能内置的温度传感器。
- Bit 2:0 (CLKSEL) = 000: 选择内部 8MHz 振荡器作为时钟源。

2. MPU6050_PWR_MGMT_2

电源管理寄存器 2，负责控制各个传感器的开关。

- 这个寄存器的高 2 位 (Bit 7:6) 是低功耗唤醒频率，默认不用管。
- 低 6 位分别是 加速度计 X、Y、Z 轴和 陀螺仪 X、Y、Z 轴的待机控制位。
- 全为 0 的意义：让 3 轴加速度计和 3 轴陀螺仪全部处于工作状态。

3. MPU6050_SMPLRT_DIV

采样率分频寄存器。

- 采样率计算公式： $\text{采样率} = 1000 / (1 + \text{SMPLRT_DIV})$ 。
- 代入计算： $1000 / (1 + 9) = 100\text{Hz}$ 。
- 意义：MPU6050 会以 每秒 100 次 的频率去刷新数据寄存器。

4. MPU6050_CONFIG

配置寄存器（主要控制数字低通滤波器 DLPF）。PS:DLPF是滤波用的

- 0x06 的二进制是 0000 0110。
- Bit 5:3 (DLPF_CFG) = 110 查数据手册可知，配置为 6 时，加速度计的带宽为 5Hz，陀螺仪的带宽为 5Hz。
 - 芯片内部会滤除 5Hz 以上的高频震动（比如电机震动），只保留人体或无人机缓慢的姿态变化。
 - 但延迟会变大。
- Bit 2:0 (EXT_SYNC_SET) = 000: 禁用外部同步信号。

5. MPU6050_GYRO_CONFIG

陀螺仪配置寄存器（设置量程）。

- 0x18 的二进制是 0001 1000。
- Bit 4:3 (FS_SEL) = 11: 设置陀螺仪满量程为 $\pm 2000^\circ/\text{s}$ 。
- 意义：这是最大的量程。意味着陀螺仪可以测量每秒旋转 2000 度的剧烈运动。
- 换算公式： $\text{角速度} (^\circ/\text{s}) = \text{原始值} / 16.4$ 。这个量程适合无人机、平衡车等运动剧烈的场景，但静态下的分辨率会稍低。

6. MPU6050_ACCEL_CONFIG

加速度计配置寄存器（设置量程）。

- 0x18 的二进制是 0001 1000。
- Bit 4:3 (AFS_SEL) = 11: 设置加速度计满量程为 $\pm 16g$ 。
- 意义：量程很大，能测量高强度的冲击和加速度。
- 换算公式： $\text{加速度} (g) = \text{原始值} / 2048$ 。

得到数据

配置完了就可以去Getdata了，不过这里建议再写一个连续读取的函数，不然待会每次读数据都要来一次完整的I2C协议。

```
uint8_t MPU6050_BurstRead(uint8_t *buf)
{
    uint8_t i;
    uint8_t ack;
    MyI2C_Start();
    MyI2C_SendByte(MPU6050_ADDR);
    ack = MyI2C_ReceiveAck();
    if(ack == 1)
    {
        MyI2C_Stop();
        return 1;
    }
    MyI2C_SendByte(MPU6050_ACCEL_XOUT_H);
    ack = MyI2C_ReceiveAck();
    if(ack == 1)
    {
        MyI2C_Stop();
        return 1;
    }
    MyI2C_Start();
    MyI2C_SendByte(MPU6050_ADDR | 1);
    ack = MyI2C_ReceiveAck();
    if(ack == 1)
    {
        MyI2C_Stop();
        return 1;
    }
    for(i=0;i<14;i++)//连续读十四个字节的数据
    {
        buf[i] = MyI2C_ReceiveByte();
        if(i < 13)
            MyI2C_SendAck(0);
        else
            MyI2C_SendAck(1);//终止读取告诉从机我不读了
    }
    MyI2C_Stop();
    return 0;
}

uint8_t MPU6050_GetData(int16_t *AccX, int16_t *AccY, int16_t *AccZ, int16_t
*GyroX, int16_t *GyroY, int16_t *GyroZ)
{
    uint8_t buf[14];
    uint8_t ret = MPU6050_BurstRead(buf);
    if(ret != 0)
    {
        return 1;
    }
}
```

```

*AccX = (int16_t)((buf[0] << 8) | buf[1]);
*AccY = (int16_t)((buf[2] << 8) | buf[3]);
*AccZ = (int16_t)((buf[4] << 8) | buf[5]);
*GyroX = (int16_t)((buf[8] << 8) | buf[9]);
*GyroY = (int16_t)((buf[10] << 8) | buf[11]);
*GyroZ = (int16_t)((buf[12] << 8) | buf[13]);
return 0;
}

```

首先解答**第一个问题**：数据为什么能这么算？

答：因为寄存器分别传回的是原始数据的高八位和低八位的补码形式，在接收到第一个数据后左移八位，接着再与第二个数据进行或运算就可以得到原始数据的补码形式了，再进行一次强转就行

接着解答**第二个问题**：为什么可以用数组一个个接收这些数据？

MPU6050 的数据寄存器地址是连续的，从 0x3B 开始，依次排列：

寄存器地址	数据含义	字节序号（你在 buf 中的位置）
0x3B	加速度 X 高字节	buf[0]
0x3C	加速度 X 低字节	buf[1]
0x3D	加速度 Y 高字节	buf[2]
0x3E	加速度 Y 低字节	buf[3]
0x3F	加速度 Z 高字节	buf[4]
0x40	加速度 Z 低字节	buf[5]
0x41	温度高字节	buf[6]
0x42	温度低字节	buf[7]
0x43	陀螺仪 X 高字节	buf[8]
0x44	陀螺仪 X 低字节	buf[9]
0x45	陀螺仪 Y 高字节	buf[10]
0x46	陀螺仪 Y 低字节	buf[11]
0x47	陀螺仪 Z 高字节	buf[12]
0x48	陀螺仪 Z 低字节	buf[13]

这也是我跳过了buf[6]和buf[7]的原因，因为它俩是温度寄存器的数据

最后数据还可以处理一下，**换算公式**：加速度(g) = 原始值 / 2048，角速度(°/s) = 原始值 / 16.4，我这里没搞

4.HMC5883L-3轴磁力计

MC5883L在硬件上是挂在MPU6050的“辅助 I2C 总线”上的，它的 SDA/SCL 连着 MPU6050 的 XDA/XCL 引脚，相当于MPU6050是它的桥梁。你直接利用 `I2C_Scan()` (我写的一个用于扫描I2C总线上的从机的函数,本质就是遍历地址,有应答就是有从机,返回这个地址)是读不到这个芯片的地址的，非常畜生。于是就有下面两种访问模式了：

访问方式

- **旁路模式**：配置 MPU6050 打开旁路，让 STM32 直接和它通话。
- ****主机模式**：配置 MPU6050 去读它，STM32 从 `EXT_SENS_DATA` 读结果。推荐用旁路模式，代码更简单。

下面是旁路模式开启代码,我懒得把地址再define一下了,贴两张图

```
MPU6050_WriteReg(0x6A, 0x00); // 确保关闭 MPU6050 的 I2C 主机模式
MPU6050_WriteReg(0x37, 0x02); // 设置 INT_PIN_CFG 寄存器，启用旁路模式
```

这也是和MPU6050配置上最大的一个不同。

4.15 REGISTER 55 - INT PIN / BYPASS ENABLE CONFIGURATION

INT 引脚/旁路有效 使能配置

INT_PIN_CFG

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
37	55	INT_LEVEL	INT_OPEN	LATCH_INT_EN	INT_RD_CLEAR	FSYNC_INT_LEVEL	FSYNC_INT_EN	I2C_BYPASS_EN	-

说明:

这个寄存器配置 **INT** 引脚中断信号的产生。也用于使能 **FSYNC** 引脚作为主应用处理器的中断，也使能 **I2C** 主机的 **Bypass** 模式。该位也使能时钟输出。

FSYNC_INT_EN 使能 **FSYNC** 引脚作为主应用处理器中断。**FSYNC_INT_LEVEL** 指定的过渡有效电平将触发中断。中断状态可以从 **I2C** 主机状态寄存器（寄存器 54）**PASS_THROUGH** 位读取。

当 **I2C_BYPASS_EN** 等于 1 且 **I2C_MST_EN**（寄存器 106 第 5 位）等于 0，主应用处理器可以直接访问 **MPU-60X0** 的辅助 **I2C** 总线。当该位等于 0，不管 **I2C_MST_EN** 什么状态，主应用处理器都不能直接访问 **MPU-60X0** 的辅助 **I2C** 总线。

更多关于 **Bypass** 模式的信息，请参阅 **MPU-6000/6050** 产品特性文档第 7.11 和 7.13 节。

参数:

INT_LEVEL	该位等于 0，INT 引脚的逻辑电平为高电平。 该位等于 1，INT 引脚的逻辑电平为低电平。
INT_OPEN	该位等于 0，the INT pin is configured as push-pull. 该位等于 1，the INT pin is configured as open drain.
LATCH_INT_EN	该位等于 0，INT 引脚产生 50us 脉冲。 该位等于 1，INT 引脚保持高电平直到中断被清除。
INT_RD_CLEAR	该位等于 0，读取 INT_STATUS (寄存器 58)中断状态位清除。 该位等于 1，任何读取中断状态位清除。
FSYNC_INT_LEVEL	该位等于 0，FSYNC 引脚的逻辑电平为高(作为主处理器中断)。 该位等于 1，FSYNC 引脚的逻辑电平为低(作为主处理器中断)。
FSYNC_INT_EN	该位等于 0，该位失能 FSYNC 引脚从主处理器产生中断。 该位等于 1，该位使能 FSYNC 引脚作为主处理器中断。
I2C_BYPASS_EN	该位等于 1 且 I2C_MST_EN (寄存器 106 bit[5]) 等于 0，主应用处理器可以直接访问 MPU-60X0 的辅助 I2C 总线。 该位等于 0，无论 I2C_MST_EN (Register 106 bit[5]) 什么状态，主应用处理器不可以直接访问 MPU-60X0 的辅助 I2C 总线。

4.29 REGISTER 106 - USER CONTROL

用户配置

USER_CTRL

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
6A	106	-	FIFO_EN	I2C_MST_EN	I2C_IF_DIS	-	FIFO_RESET	I2C_MST_RESET	SIG_COND_RESET

说明:

这个寄存器允许用户使能或使能 **FIFO** 缓冲区, **I2C** 主机模式和主要 **I2C** 接口。 **FIFO** 缓冲区, **I2C** 主机, 传感器信号通道和传感器寄存器也可以使用这个寄存器复位。

当 **I2C_MST_EN** 置 1, **I2C** 主机模式使能。在这个模式下, **MPU-60X0** 作为 **I2C** 主机通过辅助 **I2C** 总线连接外部传感器从机设备。当该位清 0, 辅助 **I2C** 总线(**AUX_DA** 和 **AUX_CL**) 逻辑上由主 **I2C** 总线 (**SDA** 和 **SCL**) 驱动。这是使能盘路模式 (**Bypass Mode**) 的先决条件。更多关于旁路模式的信息, 请参阅寄存器 55。

MPU-6000: 当 **I2C_IF_DIS** 置 1, 主 **SPI** 接口的使能取代主 **I2C** 接口。

MPU-6050: 一般 **I2C_IF_DIS** 写 0。

当复位位(reset bits) (**FIFO_RESET**, **I2C_MST_RESET**, and **SIG_COND_RESET**) 置 1, 这些复位位会产生一个复位并清 0。

参数:

FIFO_EN

该位置 1, 使能 **FIFO** 操作。

该位清 0, 失能 **FIFO** 缓冲区。 **FIFO** 缓冲区不能读取或写入。

FIFO 缓冲区状态不能改变除非 **MPU-60X0** 重新上电。

I2C_MST_EN

该位置 1, 使能 **I2C** 主机模式。

该位清 0, 辅助 **I2C** 总线 (**AUX_DA** 和 **AUX_CL**) 逻辑上由主 **I2C** 总线 (**SDA** 和 **SCL**) 驱动。

I2C_IF_DIS

MPU-6000:该位置 1, 失能主 **I2C** 接口并使能 **SPI** 接口

MPU-6050: 一般该位写 0。

FIFO_RESET

当 **FIFO_EN**=0 该位置 1, 复位 **FIFO** 缓冲区。复位后该位自动清 0。

I2C_MST_RESET

当 **I2C_MST_EN**=0 该位置 1, 复位 **I2C** 主机。复位后该位自动清 0。

SIG_COND_RESET

该位置 1, 复位所有传感器的信号通道 (陀螺仪, 加速度计和温度传感器)。这个操作会清除传感器寄存器。复位后该位自动清 0。如果只想复位信号通道(不复位传感器寄存器), 请使用寄存器 104, **SIGNAL_PATH_RESET**。

这两张图片重点看**I2C_BYPASS_EN**和**I2C_MST_EN**这两个寄存器就行

初始化

```
void HMC5883L_Init()
{
    MyI2C_Init();
    MPU6050_WriteReg(0x6A, 0x00); // 确保关闭 MPU6050 的 I2C 主机模式
    MPU6050_WriteReg(0x37, 0x02); // 设置 INT_PIN_CFG 寄存器，启用旁路模式
    // 配置A: 8次平均, 15Hz输出, 正常测量
    HMC5883L_WriteReg(HMC5883L_CRA, 0x70);
    // 配置B: 增益 ±1.3Ga (灵敏度1090 LSB/Gauss)
    HMC5883L_WriteReg(HMC5883L_CRB, 0x20);
    // 模式: 连续测量模式
    HMC5883L_WriteReg(HMC5883L_MODE, 0x00);
}
```

1. 配置寄存器 A

决定多久测一次和测多少次取平均。

位	名称	我设定的值	含义
7:5	MA (平均采样数)	011 (十进制 3)	8 次平均 。意味着每输出 1 个数据，芯片内部会连续测 8 次再取平均值。这能显著降低噪声，但会稍微增加功耗。
4:2	DO (输出速率)	100 (十进制 4)	15Hz 输出频率 。即每秒输出 15 次数据。
1:0	MS (测量模式)	00	正常测量模式 。不进行偏置校准，就是我们日常使用的最基本模式。

2. 配置寄存器 B

决定测量量程和灵敏度

位	名称	你的值	含义
7:5	GN (增益)	001 (十进制 1)	±1.3 Ga 量程 。Gauss 是磁场单位。地球表面磁场大约在 0.25 ~ 0.65 Ga 之间。
4:0	保留	00000	没用到，必须保持为 0。

- 为什么要用 ±1.3 Ga?
这是 HMC5883L 量程中**分辨率最高**的一档。灵敏度可达 **1090 LSB/Gauss**。
因为地球磁场很弱，用 ±1.3 Ga 刚好能覆盖地球磁场，且分辨率高。
注意事项：如果你把模块靠近强磁铁（比如电机、大喇叭），磁场超过 1.3 Ga，数据就会“饱和”和“溢出”，变成固定的最大/最小值。此时就必须把量程改成 ±4.7 Ga

3. 模式寄存器

作用：决定“芯片是否持续工作”。这个寄存器 8 位中只有最低两位（Bit 1:0）有用：

位	名称	你的值	含义
1:0	MD（模式）	00	**连续测量模式。芯片会自动以 CRA 设定的 15Hz 频率持续测量并刷新数据寄存器。
7:2	保留	000000	必须保持为 0。

- 0x00（连续）：正常工作，数据一直刷新。
- 0x01（单次）：测一次就休眠。必须每次读数据前重新写一次 0x01 触发测量，否则读到的永远是最初的那个死值。
- 0x03（空闲）：完全休眠，数据不更新。

得到数据

和MPU6050完全类似,自己写一下试试。

5.MS5611

它和前面两者其实区别还蛮大的，具体如下表：

芯片	通信模型	读数据流程
MPU6050	寄存器式	Start -> 写地址 -> 写寄存器地址 -> Restart -> 读地址 -> 读数
MS5611	命令式	Start -> 写地址 -> 写命令 -> Stop -> Start -> 读地址 -> 读数

它仅仅只有五条基本命令：

COMMANDS

The MS5611-01BA has only five basic commands:

- Reset
- Read PROM (128 bit of calibration words)
- D1 conversion
- D2 conversion
- Read ADC result (24 bit pressure / temperature)

1. 复位

2. 读128位校准字-----命令 0xA2 ~ 0xAC

什么是校准字?

答：芯片出厂时，每颗芯片的传感器都有微小差异，厂家把这6个校准系数（C1~C6）烧进了PROM里。图片里写着128 bit，但只有前6个（C1~C6）有用，剩下的包含CRC校验和保留位，通常不用管。

3. 进行一次气压（D1）的ADC转换

4. 进行一次温度（D2）的ADC转换

ADC是什么?

模数转换器，用于将电压信号转化为数字信号，比如当气压变化时，传感器内部的电容值会变化，转化为微弱的电压变化，ADC就负责把这个微弱的电压，转换成一个精确的数字。STM32里也有这个玩意可以自己配置，详情请去江科大学习。

5. 读ADC转换结果

读写代码

```
static void MS5611_WriteCmd(uint8_t cmd)
{
    MyI2C_Start();
    MyI2C_SendByte(MS5611_ADDR);
    MyI2C_ReceiveAck();
    MyI2C_SendByte(cmd); //这里直接写命令就行
    MyI2C_ReceiveAck();
    MyI2C_Stop();
}

static void MS5611_ReadBytes(uint8_t *buf, uint8_t len)
{
    MyI2C_Start();
    MyI2C_SendByte(MS5611_ADDR | 1); // 读地址
    MyI2C_ReceiveAck();
    for (uint8_t i = 0; i < len; i++)
    {
        buf[i] = MyI2C_ReceiveByte();
        if (i == len - 1) MyI2C_SendAck(1);
        else MyI2C_SendAck(0);
    }
    MyI2C_Stop();
}
```

初始化

它的初始化和另外俩不同不用配置寄存器，很方便，我们只需要把PROM读出来就行

```

uint16_t C[7];
void MS5611_Init()
{
    MS5611_WriteCmd(MS5611_CMD_RESET);
    uint8_t buf[2];
    for (uint8_t i = 1; i <= 6; i++)
    {
        MS5611_WriteCmd(MS5611_CMD_PROM_BASE + i * 2);
        MS5611_ReadBytes(buf, 2);
        C[i] = (buf[0] << 8) | buf[1];
    }
}

```

读数据

不解释了自己看看代码

```

void MS5611_GetData(float *pressure, float *temperature)
{
    uint8_t buf[3];
    uint32_t D1,D2;

    MS5611_WriteCmd(MS5611_CMD_D1_OSR4096);
    Delay_ms(10); // 等待转换
    MS5611_WriteCmd(MS5611_CMD_ADC_READ);
    MS5611_ReadBytes(buf,3);
    D1 = ((uint32_t)buf[0]<<16) | ((uint32_t)buf[1]<<8) | buf[2];

    MS5611_WriteCmd(MS5611_CMD_D2_OSR4096);
    Delay_ms(10);
    MS5611_WriteCmd(MS5611_CMD_ADC_READ);
    MS5611_ReadBytes(buf, 3);
    D2 = ((uint32_t)buf[0] << 16) | ((uint32_t)buf[1] << 8) | buf[2];

    //一阶补偿计算(看不懂,直接抄的)
    int32_t dT = D2 - ((uint32_t)C[5] << 8);
    int32_t TEMP = 2000 + ((int64_t)dT * C[6]) / 8388608;

    int64_t OFF = ((int64_t)C[2] << 16) + ((int64_t)C[4] * dT) / 128;
    int64_t SENS = ((int64_t)C[1] << 15) + ((int64_t)C[3] * dT) / 256;

    int32_t P = ((int64_t)D1 * SENS / 2097152 - OFF) / 32768;

    *temperature = TEMP / 100.0f; // 单位 °C
    *pressure = P / 100.0f; // 单位 mbar, 如果想输出 Pa, 可以再乘以 100
}

```

完工!

